当演化路由失效：
结构化决策模型中的动作互补性与效用预测

中文审阅版

2026 年 11 月

摘要

结构化决策模型（如 Jev）把一次语言模型调用变成带类型的概率判断：二元 Noul 判定、候选 Choice 选择或有序 Score 评分。这一接口让推理期路由显得很有吸引力：先问一个廉价的 Noul 问题，只有当第一个答案看起来不确定时，才购买第二个、以不同方式表述的 Choice 视角。我们用冻结的开源模型检验这一直觉。在 CodeXGLUE 缺陷检测、FinDVer 金融主张核验和 ProcessBench 推理错误定位三个任务上，使用 Laya 0.4.16 与 Vela 2.0 0.3B，把遗传算法、进化策略和 NSGA-II 与固定策略、预算匹配的随机搜索进行比较，并测量路由真正需要的两个量：逐样本的动作互补性，以及第二个动作增量效用的可预测性。总互补机会确实存在——逐行 oracle 相对更好的固定动作最多可提升 23.3 个准确率点——但净修复稀疏且依赖数据池：四个 confirm 单元中只有一个的准确率区间为正，其 macro-F1 区间仍然跨零。概率特征在某些池中能把可修复行排到随机水平之上（折外 AUROC 最高 0.92），但冻结阈值在留出数据上不带来收益。遗传算法在 60 组配对比较中从未胜过随机搜索；它对固定切换的 5 次显著胜利全部出现在训练池，在 dev 与 confirm 上消失。21 次搜索中有 15 次在第 0 代就停止改进。温度缩放能降低校准误差，却无法创造路由信号，因为单调重校准不改变带策略类。路由价值需要互补动作与可部署的增量动作价值估计；校准置信度本身二者都不提供。

1 引言

一位金融分析员问模型：这句话能否被一份财报支持？一位安全工程师问：这段 C 函数是否含有漏洞？一位数学家问：第一个无效步骤出现在哪里？这些场景需要的是一个可以据以分支的决策，而不是一段必须由人再解读的文字。System One 类模型直接回应这一需求。给定一个状态和一个带类型的问题，Jev 返回二元 Noul、候选 Choice 或带概率信息的有序 Score，调用程序无需第二次解释就能据此行动 (TypeSafe AI, 2026; Deußer et al., 2026)。Laya 与 Vela 以开放权重提供类似接口 (Laya Project, 2026; vllm-sr, 2026)。不确定性由此成为软件可以直接读取的一等值。

这个值提示了一条显而易见的控制策略：先问廉价的二元问题，检查返回的概率，只有当第一个答案看起来不稳时，才把第二次调用花在表述不同的 Choice 问题上。策略容易实现，部署时不需要标签，也符合从业者使用置信度阈值的习惯。但它依赖一个很少被明说的假设：当前决策所附的概率，能告诉我们另一个决策的价值。模型可以不确定却仍然正确，也可以很自

信却正要从正确的 Noul 答案切换到错误的 Choice 答案。路由关心的不是 P(当前答案正确)，而是替代动作的增量效用 $U(\text{Choice} \mid x) - U(\text{Noul} \mid x)$。

这两个量容易被混为一谈，因为它们是同一标签集上的概率，却由不同的监督信号学得。RLCD 是 Jev 背后的训练配方，也是 Laya 文档所描述的配方，它用严格适当评分规则优化所报告分布的诚实性 (TypeSafe AI, 2026; Laya Project, 2026; Gneiting and Raftery, 2007)。它教会模型在约七成正确时说 0.7。这一目标从未要求模型预测：换一个 Choice 问题是否会得到更好的答案。因此，一个校准良好的输出头完全可以与一个没有可用信号的路由器共存。级联继承了这个问题：Jitkrittum et al. (2023) 证明，基于置信度的推迟只有在关于下游误差与上游置信度关系的严格假设下才最优；专门化的下游误差、标签噪声或分布偏移都可能破坏该规则。

我们测量这一区分，而不是假设它。两个结构化决策模型 Laya 0.4.16 与 Vela 2.0 0.3B 被冻结，并在三个具有不同决策接口的任务上研究推理期路由：CodeXGLUE 缺陷检测 (Lu et al., 2021)、FinDVer 金融主张核验 (Zhao et al., 2024) 与 ProcessBench 推理错误定位 (Zheng et al., 2025)。在二元任务上，动作集为 {Noul, Choice}：策略总是先支付 Noul，并可能再次支付 Choice。在 ProcessBench 上，策略决定是回答 top-1 步骤还是弃权，依据候选概率与 top-1/top-2 间隔。带策略分别用遗传算法（GA）、进化策略（ES）与 NSGA-II 搜索，并在相同数量的唯一候选评估下与固定策略及随机搜索比较。所有输出在 CPU 上生成一次，并从不可变缓存回放，因此每次比较看到的行完全相同。

设计把问题拆成三个。RQ1 问动作集是否含有净互补性：第二个动作修复的行是否多于它破坏的行？RQ2 问第二个动作的增量效用能否在它执行之前，由当时可用的信号预测？RQ3 问演化搜索能否把已有信号转化为在留出数据上胜过固定与随机基线的策略。RQ1 为负会让 RQ3 失去意义；RQ1 为正而 RQ2 为负，则指向表示或监督问题，而不是优化器问题。两个边界检查——跨任务迁移与按置信度分层的故障图——检验结论是否只是某个池或某个配置的产物。

结果一致为负，而失败的方式提供了信息。总互补性真实存在：在 CodeXGLUE 与 Laya 上，逐行选择更好动作的 oracle 比更好的固定动作高 23.3 个准确率点，Choice 修复了 14 个 Noul 错误。但它同时破坏了 15 个正确答案，因此训练池上的净修复为 -1。留出池并不能稳定这一图景。只有一个 confirm 单元（FinDVer 与 Laya）的净修复准确率区间为正，但其 macro-F1 区间跨零，且更好的固定动作在 dev 与 confirm 之间发生反转。效用预测重复了同样的模式：折外 AUROC 在 FinDVer 与 Vela 上达到 0.92，但在 dev 上选出的阈值在 confirm 上产生负净效用；CodeXGLUE 与 Laya 上最好的 dev 策略退化为对每一行都切换。搜索也救不了信号：GA 在 60 组配对 macro-F1 比较中没有一次胜过随机搜索；它对固定切换的 5 次显著胜利全部发生在 FinDVer/Vela 训练池；NSGA-II 的部署在 6 个单元中显著差于固定基线。校准很差——confirm 的 ECE 从 0.09 到 0.29——但温度缩放让路由结果不变或更差，因为单调重校准无法创造带策略类原本表达不了的信号。

本文的贡献是把路由价值分解为动作互补性、增量效用可预测性与搜索有效性三个诊断量，并在冻结的结构化决策模型上测量，而不是从端任务准确率反推。我们报告一个预注册的负结果：在五个种子、三个池、四个任务—模型单元上使用配对分组自助法区间，并指出表面胜利只存在于训练池。我们还表明校准与动作价值预测会分离，并把失败定位在动作集与监督目标，而不是优化器容量。

2 背景与相关工作

结构化决策模型用带类型的问题和带类型的答案取代自由文本生成。TypeSafe 的商业 System One 模型 Jev 接收一个状态和一组问题，返回 Noul（二元判断）、Choice（在命名候选中选择）或 Score（有序评分）及相应概率 (TypeSafe AI, 2026)。Deußer et al. (2026) 在 37 个数据集上评测 Jev，Guo et al. (2026) 把它用于对齐失败检测；两者都把该模型视为通用决策接口，而不是文本生成器。TypeSafe 把训练目标描述为 Reinforcement Learning for Calibrated Decisions（RLCD），用严格适当评分规则优化所报告的分布，使概率携带 argmax 之外的信息；完整配方未公开。Gao and Wang (2026) 提供了独立的开源实现。RLCD 这一名称也出现在 Yang et al. (2023) 的 Reinforcement Learning from Contrast Distillation 中；两者无关，本文只在“校准决策”意义上使用 RLCD。

我们运行的两个模型在骨干与监督上不同，但共享接口。Laya 0.4.16 是带决策头的 ModernBERT-large 编码器，约 4.21 亿参数，在一次非自回归前向中支持 Noul、Choice 与 Score；它是最接近 RLCD 描述的开源模型 (Laya Project, 2026; Laya Studio, 2026)。Vela 2.0 0.3B 是 22 层 ModernBERT 编码器，上下文窗口 8,192 个 token；它提供同样三种动作以及 Span 与 Set，并从 Decision-1.0-Kai 的 Choice 路径初始化，使用任务特定损失与路由数据 (vllm-sr, 2026; Warner et al., 2024)。Vela 的模型卡没有把配方称为 RLCD，因此我们把 Laya 视为开放 RLCD 模型，把 Vela 视为接口匹配但训练路线不同的对照（附录 A）。

校准是审视这些概率的自然第一视角。神经网络常常过度自信，温度缩放或其他后处理映射可以在不改变 argmax 的情况下降低期望校准误差 (Guo et al., 2017)。语言模型也能部分识别自身错误，这使置信度成为看似合理的路由特征 (Kadavath et al., 2022)；适当评分规则则形式化了为什么所报告的概率应当因诚实而非仅因尖锐而得到奖励 (Gneiting and Raftery, 2007)。这些工作关心的都是模型已经给出的答案的概率，而不是该概率能否排序另一个问题类型的反事实价值。我们测量当前答案正确性的 ECE 与 Brier，再测量同一概率对“替代动作可修复行”的排序能力。

级联与路由器在不同模型或动作之间分配计算。经典级联尽早拒绝简单样本，把困难样本送给更强的分类器 (Viola and Jones, 2001)；现代系统学习何时调用更大的语言模型 (Chen et al., 2024; Aggarwal et al., 2024; Ong et al., 2024)、采样多少候选 (Ding et al., 2025)，或如何把路由与级联合并为一个目标 (Dekoninck et al., 2025)。理论上的警告来自 Jitkrittum et al. (2023)：基于置信度的推迟只有在 downstream 误差与 upstream 置信度足够紧密相关时才最优，并且会在专门化的下游误差、标签噪声或分布偏移下失效。Damani et al. (2025) 主张计算应跟随期望收益，而不是表面难度。我们的实验在一个结构化决策模型中具体化了这一警告：第二个动作是向同一个冻结网络提出的不同问题，而不是更大的模型，其价值取决于第一个概率并未编码的误差结构。对 ProcessBench，选择性预测提供了弃权框架：Chow (1970) 推导了已知分布下的最优拒绝规则，Geifman and El-Yaniv (2017)、Kamath et al. (2020) 与 Xin et al. (2021) 研究了选择性分类、问答以及覆盖率与条件准确率的分离。

当策略是一个小型不可微对象、目标又是缓存回放时，演化方法是合理选择。遗传算法、进化策略与 NSGA-II 覆盖单目标与 Pareto 形式 (Deb et al., 2002; Hansen and Ostermeier, 2001; Stanley and Miikkulainen, 2002)，正则化演化也被用于架构搜索 (Real et al., 2019; Elsken et al., 2019)。在 NLP 中，EvoPrompt 与 Promptbreeder 让语言模型参与循环，演化提示或变异指

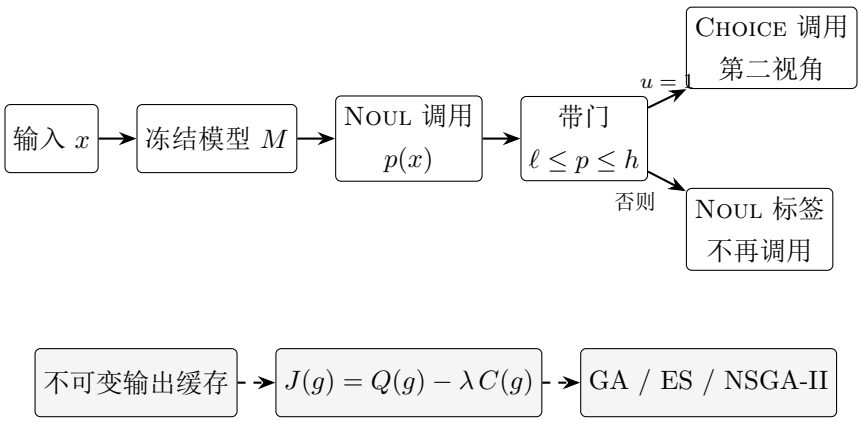


图 1: 路由设定。冻结模型回答第一个 NOUL 问题；一个两参数门决定是否向同一模型提出第二个 CHOICE 问题。搜索完全在缓存输出上进行：候选基因组通过回放缓存评分，优化器只看到标量适应度 J。虚线路径离线，实线路径是部署策略。

令 (Guo et al., 2024; Fernando et al., 2024)。这些系统搜索的是改变模型所见内容的文本；我们固定模型与问题模板，只搜索一个两参数门，决定是否提出第二个带类型问题。提示搜索可以改进单次调用，而我们的搜索只能利用两个固定动作之间已经存在的互补性。因此，平坦的搜索地形是关于动作集的发现，而不只是关于优化器强度的发现。

我们的协议遵循 Henderson et al. (2018)、Bouthillier et al. (2021) 与 Dodge et al. (2019) 的方差感知报告实践：在查看 confirm 之前固定动作集、模板与预算；报告每个种子级区间，而不是只报告最好单元；把锁定测试标为未运行，而不是用 confirm 数字替代。

3 方法与决策规则

我们研究一个冻结模型 M 与固定动作集 $\mathcal{A}$。输入 x 由任务适配器渲染为状态字符串与带类型问题；模型返回经过模式校验的决策及其概率。策略只读取第一次调用，并决定是否发出第二次调用。模型权重、问题模板与候选标签集在搜索过程中从不修改。这样就把推理期控制与在路由目标上微调骨干区分开来。

对二元任务 CodeXGLUE 与 FinDVer，第一个动作是 NOUL：模型回答一个是/否问题并返回 $p(x) = P(\text{真} \mid x)$。第二个动作是 CHOICE：模型在两个命名标签之间选择。在 CodeXGLUE 中，问题是函数是否含有漏洞，以及应标为 `secure` 还是 `vulnerable`；在 FinDVer 中，问题是主张是否被支持，以及应标为 `supported` 还是 `not supported`。级联总是先支付 NOUL，只有当门打开时才支付 CHOICE。策略是基因组 $g_{\text{cas}} = (\ell, h, u)$，其中 $\ell, h \in [0, 1]$，$u \in \{0, 1\}$：

$$\pi_g(x) = \begin{cases} \textsc{Choice}(x), & u = 1 \wedge \ell \le p(x) \le h, \\ \textsc{Noul}(x), & \text{否则}. \end{cases} \tag{1}$$

门使用正类概率而不是对称置信度，因为 NOUL 问题有一个指定的真标签。基因组修复会把 ℓ 与 h 裁剪到 $[0, 1]$，在 $\ell > h$ 时交换二者，并把 u 映射到 $\{0, 1\}$。两个固定策略是特例：$u = 0$ 是仅 NOUL；$u = 1, \ell = 0, h = 1$ 是固定仅切换，它仍然支付第一次 NOUL 调用，因此每行成本为两次调用。

ProcessBench 的动作空间不同。模型返回候选步骤的排序列表，策略决定是用 top-1 步骤回答还是弃权。基因组 $g_{\text{abs}} = (t, v, m)$，其中 $t, m \in [0, 1]$，$v \in \{0, 1\}$，使用 top-1 概率 $s_1(x)$ 与 top-1/top-2 间隔 $\Delta(x)$：

$$\text{abstain}(x) = \begin{cases} \text{真}, & s_1(x) < t, \\ \text{真}, & v = 1 \wedge \Delta(x) < m, \\ \text{假}, & \text{否则}. \end{cases} \tag{2}$$

当所需置信度字段缺失时，评估器按失败关闭处理：该行被强制回答，并计为错误，而不是被悄悄丢弃。我们报告覆盖率（被回答行的比例）与选择性准确率（被回答行上的准确率）。两个基因组从不共享动作集或成本模型，因此我们从不合并它们的准确率数字。

适应度把任务质量与第二次调用的成本结合起来：

$$J(g) = Q(g) - \lambda C(g), \tag{3}$$

其中 Q 在二元任务上是 macro-F1，在 ProcessBench 上是步骤准确率；$C(g)$ 是每行模型调用均值减一；$\lambda = 0.05$ 在搜索前固定。Macro-F1 作为主指标，因为两个二元数据集都类别不平衡；准确率同时报告。第一次 NOUL 调用计入每个策略，包括仅 NOUL，因此成本项隔离的是增量调用，而不是总推理账单。模型输出只生成一次，并从不可变 JSON 缓存回放，这使候选评估确定且廉价。我们测量原生动作的端到端延迟，但不为每个搜索出的策略声称延迟。

GA 使用锦标赛选择、均匀交叉、高斯变异与 u 的位翻转。ES 是 $(\mu + \lambda)$ 策略，变异两个端点并固定 $u = 1$。NSGA-II 维护非支配前沿，在二元任务上优化准确率与负调用成本，在 ProcessBench 上优化选择性准确率与覆盖率；由于其质量目标是准确率而不是 macro-F1，我们单独保留其区间。主预算是 24 次唯一基因组评估，种子 17、23、41；五种子扩展加入 59 与 73。一个探针把 CodeXGLUE/Vela 的预算提高到 48。随机搜索使用相同预算与缓存。

策略类的一个性质塑造了校准实验。门只通过区间 $\ell \le p \le h$ 依赖 $p(x)$。任何严格单调变换——例如温度缩放——都把区间映射为另一个区间，并在端点重参数化意义下诱导同一组被切换行。因此，单调重校准无法改变固定池上可达的最优带策略。它可以移动行在带之间的位置，但无法创造原始排序本就不包含的“可修复行 vs 有害行”区分。因此校准与路由分开评估。

4 实验设计

实验使用三个任务，其标签与分组结构在任何策略搜索之前就已固定。CodeXGLUE 缺陷检测提供 21,854 个训练、2,732 个 dev 与 2,732 个测试函数；控制器池只从官方训练划分中抽取，并按函数文本的精确哈希分组，使相同函数不能跨越内部 `search_train`、`selection_dev` 与 `confirm` 池 (Lu et al., 2021; Zhou et al., 2019)。每个内部池含 60 个函数。这控制了重复函数泄漏，但不是项目级留出。FinDVer 提供带 `supported/not supported` 标签的金融主张；其池按报告分组，分别含 60、59 与 44 条主张 (Zhao et al., 2024)。官方 FinDVer 测试标签不用于控制器搜索。ProcessBench 含 2,842 道数学题上的 3,400 条标注记录；其池按问题分组，每池 60 条 (Zheng et al., 2025; Cobbe et al., 2021; Lightman et al., 2024; Uesato et al., 2022)。任务是找出最早错误步骤，或确认推理链正确。

所有推理在一台 Windows 机器上进行，CPU-only PyTorch，FP32，两个线程，同一时间只驻留一个模型。Laya 固定到源码修订 `b9fe351` 与检查点修订 `7b928d8`；Vela 固定到修订 `d6f03aa`。峰值常驻内存低于 2.2 GB。`search_dev` 上的原生单动作基线见附录 A：CodeXGLUE 上 Laya 的准确率为 0.450/0.650（NOUL/CHOICE），Vela 为 0.633/0.667；FinDVer 上 Laya 为 0.712/0.644，Vela 为 0.458/0.458；ProcessBench 上两个模型的 CHOICE 准确率均为 0.10。两种表示的每次调用延迟接近（约 2.2–2.8 s），因此式 3 的成本项使用调用次数，而不是模型特定的延迟模型。原生数字已经包含一个警告：更好的表示取决于任务与模型，而在 FinDVer 上 Vela 的两个动作在总体层面无法区分。

E1 在不训练路由器的情况下测量动作互补性。对每个任务—模型单元，我们先在 `search_train` 上取更好的固定动作并冻结该方向；然后统计替代动作修复基础错误的行数、破坏正确基础答案的行数，以及二者之差。Oracle 上限是逐行 oracle 的准确率减去更好固定动作的准确率。由于方向在 train 上固定，留出的净修复数字没有在评估池上选择。我们以百分点报告净修复，使用分组百分位自助法（5,000 次重采样），并与同一批行上固定 CHOICE 与固定 NOUL 的 macro-F1 差配对。ProcessBench 只有一个被评估的表示，因此其上限按构造为零，被排除在二元互补性表之外。

E2 问第二个动作的增量效用能否在该动作执行之前预测。预测目标是切换是否会修复当前 NOUL 错误。特征在决策时刻可部署：p、$|p-0.5|$ 与二元熵 $-p\log p-(1-p)\log(1-p)$；不使用任何 CHOICE 输出、金标签或下游特征。我们比较逻辑回归、浅层决策树、单置信度阈值与从不切换的多数基线。训练使用分组感知的三折折外预测；预测器与阈值随后在 `selection_dev` 上按净效用联合选择，冻结，并在 `confirm` 上评估一次。净效用是（修复 − 退化）除以行数；成本调整版本每次额外调用减去 $\lambda = 0.05$。对每一行都切换的冠军策略会被保留，因为它揭示预测器是否比固定切换多提供了任何东西。

E3 在匹配的评估预算下比较优化器。GA、ES 与 NSGA-II 在缓存池上运行，使用 24 次唯一基因组评估与种子 17、23、41；五种子扩展加入 59 与 73。随机搜索获得相同预算。主要部署比较是预注册的：在 `search_train` 上选出的 GA 基因组被冻结，并在 `confirm` 上与固定仅切换比较，使用配对分组自助法 macro-F1 区间。其他单元级比较都是探索性的，报告效应估计与区间，而不是显著性断言。E4 检查搜索轨迹本身：每代最优适应度、实际评估的唯一基因组数、保持适应度不变的单基因变异比例、档案中 train 与 dev 适应度的秩相关，以及把 CodeXGLUE/Vela 的预算上限从 24 提高到 48 的效果。这些诊断使用已有日志与缓存，不增加模型调用。

E5 检查两个边界。迁移实验冻结每个源任务基因组，并将其应用到所有目标任务与池，不重新搜索；对角线单元是同任务部署检查，非对角线单元是真正的迁移。故障图按任务、模型与由 NOUL 概率或候选间隔导出的置信带划分行，并报告每个单元在 discover、dev 与 confirm 上的错误严重度。一个单元在 discover 池上至少有八行时合格；当 dev 或 confirm 严重度与 discover 严重度之差在 0.15 以内时，该单元被确认。

读表时有两个统计细节很重要。所有二元比较使用配对分组自助法，5,000 次重采样与固定随机状态 (Efron and Tibshirani, 1993)；重采样单位是组（函数哈希或报告），而不是行，因此报告内的依赖得以保留。我们报告修复/有害计数的 McNemar 精确检验 (McNemar, 1947) 与单元内严重度的 Wilson 区间 (Wilson, 1927)，但不基于单一种子或单个池宣称显著。锁定测试

表 1: 动作互补性，train 选出的方向跨池冻结（每单元 $n = 60$）。上限是逐行 oracle 相对 `search_train` 上更好固定动作的准确率增益；修复/有害统计替代动作修复或破坏基础决策的行数。净修复是配对差值（百分点，替代动作减基础动作），dev 与 confirm 上给出 95% 分组自助法区间。上限是上界，不是可部署增益。

任务 / 模型	方向	上限	修复/有害	dev 净 (95% CI)	confirm 净 (95% CI)
CodeXGLUE / Laya	Noul→Choice	0.233	14/15	+20.0 [+1.7, +38.3]	+1.7 [-18.3, +20.0]
CodeXGLUE / Vela	Switch→Noul	0.067	4/10	-3.3 [-15.0, +8.3]	+0.0 [-15.0, +15.0]
FinDVer / Laya	Noul→Choice	0.083	5/6	-6.8 [-23.3, +6.0]	+13.6 [+2.0, +24.6]
FinDVer / Vela	Switch→Noul	0.100	6/7	+0.0 [-9.1, +11.5]	-2.3 [-10.3, +5.0]

尚未运行。confirm 池在搜索与阈值选择中留出，但在分析期间已被查看；锁定评估需要先冻结模板、策略类与统计量。

5 结果与讨论

第一个问题是动作集是否有值得路由的东西。表 1 与图 2（附录 A）在逐行层面回答这个问题。总互补性并不稀缺。在 CodeXGLUE 与 Laya 上，Choice 修复了 60 个 Noul 错误中的 14 个，逐行 oracle 达到 0.783 准确率，而更好的固定动作为 0.550，上限为 0.233。同一单元也说明上限为什么是一个糟糕的承诺：Choice 破坏了 15 个正确的 Noul 答案，因此训练池上的净计数为 -1，净修复区间跨零。其他三个二元单元的总机会更小（上限 0.067–0.100），净计数在 -6 到 $+1$ 之间。在四个单元中，训练池从未产生与零可区分的净修复。

留出池没有把总机会转化为稳定的净收益。CodeXGLUE/Laya 单元在 `selection_dev` 上提高 20.0 点（$[+1.7, +38.3]$），但同一冻结方向在 confirm 上只提高 1.7 点（$[-18.3, +20.0]$）。这一差距是选择池效应，而不是路由效应：dev 正是选择方向的那个池。FinDVer 与 Laya 是检验叙事的例外。其 confirm 净修复为 +13.6 点 $[+2.0, +24.6]$，固定动作准确率差为 +0.136 $[+0.020, +0.246]$。然而相应的 macro-F1 差为 +0.111，区间跨零，且更好的固定动作从 dev 上的 Noul 反转为 confirm 上的 Choice。因此该单元在 confirm 上含有真实的准确率层面修复信号，但没有 macro-F1 收益，也没有一个 train 选出的策略本会知道去利用的方向。CodeXGLUE 与 Vela 走向相反：冻结方向是 Choice 到 Noul，train 净值为 -6，confirm 恰好为零。FinDVer 与 Vela 在 confirm 上为负（-2.3 点）。互补性存在于动作集中，但稀疏、依赖指标且依赖池。

修复出现的位置与数量同样重要。图 3（附录 A）画出修复率对 Noul 概率分箱的关系。修复集中在中间与高带，约 $p \in [0.4, 0.8]$，在最低带接近零。这与置信度级联的假设相反：阈值规则在模型不确定时切换，而这恰恰是 Choice 最不可能修复任何东西的地方，因为它能修复的行是 Noul 自信却错误的地方。这也解释了为什么校准救不了路由：重校准单调地移动概率质量，但不会把可修复行从分布中部移到低置信度尾部。

第二个问题是 Choice 的增量价值能否从第一次调用预测。表 2 报告折外排序质量与冻结的 dev 选择策略的净效用。排序质量差异很大。逻辑回归在 CodeXGLUE 与 Laya 上达到 0.674，在 FinDVer 与 Vela 上达到 0.919；决策树在 CodeXGLUE 与 Laya 上达到 0.739。在 FinDVer 与 Laya 上，两个预测器都接近随机（0.455 与 0.466），考虑到该单元只有五次训练修复，这并不奇怪。策略结果弱于排序结果。CodeXGLUE 与 Laya 上最好的 dev 策略是逻辑回归，

表 2: 增量效用预测。折外 AUROC 在 `search_train` 上使用分组感知三折交叉验证；冠军模型与阈值在 `search_dev` 上选择，冻结后用于 `confirm`。净效用为（修复 − 退化）/n，单位为百分点，括号内为计数；最后一列每次额外调用减去 $\lambda = 0.05$。

任务 / 模型	LogReg 折外	树折外	冠军	dev 净	confirm 净	confirm 成本调整
CodeXGLUE / Laya	0.674	0.739	logreg	+20.0 (23/11)	+1.7 (17/16)	-3.3
CodeXGLUE / Vela	0.716	0.646	logreg	+3.3 (7/5)	-1.7 (9/10)	-6.4
FinDVer / Laya	0.455	0.466	tree	+1.7 (1/0)	+2.3 (1/0)	+1.5
FinDVer / Vela	0.919	0.794	logreg	+3.4 (4/2)	-2.3 (0/1)	-2.6

表 3: 预注册协议下的搜索比较。计数为排除零的配对分组自助法 95% 区间数；“正”表示演化策略胜过所述基线。GA 与 ES 行在三个池、五个种子上使用 macro-F1；NSGA-II 行在 12 个 dev 与 12 个 confirm 单元上使用准确率。

比较	正	负	解读
GA 对随机	0	0	无优势 (60)
GA 对固定	5	0	仅训练池 (60)
ES 对随机	0	0	无优势 (30)
ES 对固定	4	0	一个 confirm (30)
NSGA-II 对固定	0	6	5 dev, 1 confirm

阈值对每一行都切换：dev 净 +20.0 点，confirm 净 +1.7 点，成本调整为 −3.3。FinDVer/Vela 单元尽管折外 AUROC 最高，却选出一个 dev 净 +3.4、confirm 净 −2.3 的策略（零修复，一次退化）。FinDVer 与 Laya 选择一棵只切换一行的树，在 confirm 上提高 2.3 点，这是一个建立在单次决策上的点估计。CodeXGLUE 与 Vela 在 dev 上为正（+3.3），在 confirm 上为负（−1.7）。没有任何单元能把高排序分数转化为既为正、又有足够支持度的 confirm 净效用。

校准是对基于概率的路由失败最自然的解释，因此它作为对照。表 6 与附录 A 中的图 4 表明，原始概率在 confirm 上校准很差：ECE 从 0.090 到 0.290，Brier 从 0.218 到 0.287。在 `search_train` 上拟合的温度缩放在四个单元中的三个降低 ECE（到 0.045–0.169），并在四个单元中全部降低 Brier。路由不受影响：原始概率给出的 confirm 净修复为 −5.0、0.0、+2.3 与 −2.3 点，温度缩放后为 −1.7、+1.7、0.0 与 0.0，正如第 3 节的不变性论证所预测。修复 AUROC 直接说明了这一点：四个单元分别为 0.791、0.444、0.336 与 0.095；在后两个单元中，概率把可修复行排到随机水平之下。当前答案的校准概率不是替代动作价值的校准概率。

第三个问题是搜索能否把已有信号转化为可部署策略。表 3 总结了比较。GA 对随机搜索是优化器价值最干净的检验：五个种子、四个任务—模型单元与三个池，共 60 组配对 macro-F1 比较，没有任何一组的区间排除零。GA 对固定仅切换有五个显著正单元，全部在 FinDVer/Vela 训练池，效应约为 +0.12；在 dev 与 confirm 上，40 组比较没有一组显著。预注册的部署比较——train 选出的 GA 对 confirm 上的固定仅切换——在每个种子级单元中都不显著。ES 对随机搜索在全部 30 组比较中不显著；ES 对固定切换有三个显著训练单元与一个显著 confirm 单元（CodeXGLUE/Laya，种子 17，+0.176），这是唯一的 confirm 级正结果，且未在种子上复现。NSGA-II 部署在五个 dev 单元与一个 confirm 单元中显著差于固定基线，没有正单元。搜索问题没有任何稳定的留出信号供任何优化器发现。

轨迹诊断解释了为什么更多搜索没有帮助。在 21 次记录的 GA/ES 运行中，有 15 次的最优适应度在第 0 代之后不再提高。中性变异率为 0.68–0.73，因此大多数变异产生的候选训练分数完全相同。两个 CodeXGLUE 单元的 train–dev 秩相关为 −0.56 与 −0.51，意味着在 train 上看起来更好的候选在 dev 上往往更差；FinDVer 单元为 +0.44 与 +0.99。把 CodeXGLUE/Vela 的预算从 24 提高到 48 并没有缩小差距：GA 在 48 的上限下只评估了 22、17 与 23 个唯一基因组，而随机搜索用满了 48 个。搜索不只是调参不足，而是信息不足。

边界检查问这些结论是否是某个池或任务的产物。附录 A 报告迁移表与故障图。迁移一致地没有帮助：在 32 个源—目标乘划分单元中，有六个相对目标的固定切换策略存在显著配对差异，且六个全部为负（两个对角线，四个非对角线）。故障图更细致：15 个合格单元中，13 个在 dev 上被确认，10 个在 confirm 上被确认。confirm 严重度最高的是 ProcessBench（Vela/中 0.833，Laya/中 0.875，Vela/低 0.936，Laya/高 0.895）。中置信度单元并不只是低置信度尾部：模型可以中等自信却仍然失败，而第二个表示在同一批行上也失败，因此任何基于第一个动作概率的门都无法创造修复。

综合来看，三个问题描述了一个在优化器被要求解决之前就已经失败的路由问题：RQ1 发现总互补性但没有稳定的净互补性，RQ2 发现排序信号但没有可靠的策略价值，RQ3 发现没有留出优势，因为地形平坦，而且训练排序在关键处不能预测 dev 排序。校准改进的是另一个量：模型可以为它回答的问题报告更好的概率，却仍然无法说明另一个问题是否会被回答得更好。

实践含义是工作顺序的改变。在构建路由器之前，先测量动作集在留出组上是否有净互补性，以及候选特征能否把可修复行排到随机水平之上。如果任一检查失败，更强的优化器不会有用；如果两者都通过，就可以在匹配预算下运行优化器比较，并使用预注册的 confirm 比较。

6 结论

结构化决策模型承诺：每次带类型判断所附的概率，可以决定何时值得购买第二次判断。我们在冻结的 Laya 与 Vela 模型上，跨代码缺陷检测、金融主张核验与数学推理错误定位的实验，并不支持这一承诺。对带策略的演化搜索相对固定策略或预算匹配的随机搜索没有产生留出优势，而失败在动作集、替代动作价值的可预测性以及搜索地形三处都可见。

总互补性真实存在——逐行 oracle 最多提高 23.3 个准确率点——但修复被有害切换抵消，净余额很小、依赖指标且跨池不稳定。修复集中在 Noul 概率的中高带，而置信度级联在低概率处切换。校准与动作价值预测分离：温度缩放改善 ECE 与 Brier，却不能改变行的排序；在两个 FinDVer 单元中，概率把可修复行排到随机水平之下。搜索地形平坦：大多数变异中性，大多数运行在第 0 代就停滞，因此训练排序在关键处不能预测 dev 排序。

这些限制也是结果的一部分：池很小，Noul 模板未经过审计，只运行了两个开源模型，锁定测试尚未运行。下一步是一个预测替代动作增量价值、并以留出净效用评判的监督目标。

参考文献

Pranjal Aggarwal, Aman Madaan, Yiming Yang, et al. 2024. AutoMix: Automatically mixing language models. In *Advances in Neural Information Processing Systems 37*.

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Jordan Vasiljevic, César Di Bello, et al. 2021. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3.

Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*.

C. K. Chow. 1970. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *Preprint*, arXiv:2110.14168.

Mehul Damani, Idan Shenfeld, Andi Peng, Andreea Bobu, and Jacob Andreas. 2025. Learning how hard to think: Input-adaptive allocation of LM computation. In *International Conference on Learning Representations*.

Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197.

Jasper Dekoninck, Maximilian Baader, and Martin Vechev. 2025. A unified approach to routing and cascading for LLMs. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 12987–13010.

Tobias Deußer, Rajkumar Ramamurthy, Lorenz Sparrenberg, and Rafet Sifa. 2026. Evaluating and benchmarking the System One model Jev. *Preprint*, arXiv:2609.37647.

Dujian Ding et al. 2025. BEST-Route: Adaptive LLM routing with test-time optimal compute. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 13870–13884.

Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A. Smith. 2019. Show your work: Improved reporting of experimental results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 2185–2194.

Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman and Hall/CRC.

Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2019. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21.

Chrisantha Fernando, Dylan S. Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2024. Promptbreeder: Self-referential self-improvement via prompt evolution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*.

Zhimin Gao and Pichao Wang. 2026. OpenJev-RLCD: A working RLCD implementation. *Preprint*, arXiv:2609.38850.

Yonatan Geifman and Ran El-Yaniv. 2017. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems 30*.

Tilmann Gneiting and Adrian E. Raftery. 2007. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *PMLR*, pages 1321–1330.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2024. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *International Conference on Learning Representations*.

Ruoqi Guo, Yi Liu, Gelei Deng, Yuekang Li, Lida Zhao, Yutao Wu, Simin Chen, Ying Zhang, and Leo Yu Zhang. 2026. Just ask Jev: Reinforcement learning for calibrated decisions as a zero-shot detector of AI alignment failures. *Preprint*, arXiv:2609.29429.

Nikolaus Hansen and Andreas Ostermeier. 2001. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32.

Wittawat Jitkrittum, Neha Gupta, Aditya Krishna Menon, Harikrishna Narasimhan, Ankit Singh Rawat, and Sanjiv Kumar. 2023. When does confidence-based cascade deferral suffice? In *Advances in Neural Information Processing Systems 36*.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language models (mostly) know what they know. *Preprint*, arXiv:2207.05221.

Amita Kamath, Robin Jia, and Percy Liang. 2020. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5684–5696.

Laya Project. 2026. Laya: Non-autoregressive System 1 decision engine, version 0.4.16. https://github.com/NandhaKishorM/laya/tree/b9fe351ce81e2d974bca2bf219c1e4030c0a05ea. Source revision b9fe351; accessed 2026-11-01.

Laya Studio. 2026. Proper scoring rules: How to reward honest probabilities. https://laya.studio/learn/proper-scoring-rules. Accessed 2026-11-01.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations*.

Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *NeurIPS 2021 Datasets and Benchmarks Track*.

Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

Isaac Ong, Amjad Almahairi, Vincent Wu, et al. 2024. RouteLLM: Learning to route LLMs with preference data. *Preprint*, arXiv:2406.18665.

Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. 2019. Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4780–4789.

Kenneth O. Stanley and Risto Miikkulainen. 2002. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127.

TypeSafe AI. 2026. Introducing System One models and Jev. https://typesafe.ai/blog/introducing-system-one-models-and-jev. Accessed 2026-11-01.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process- and outcome-based feedback. *Preprint*, arXiv:2211.14275.

Paul Viola and Michael Jones. 2001. Rapid object detection using a boosted cascade of simple features. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 511–518.

vllm-sr. 2026. Vela 2.0 0.3b: Model card and training details. https://huggingface.co/vllm-sr/Vela-2.0-0.3B. Model revision d6f03aa; accessed 2026-11-01.

Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, et al. 2024. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *Preprint*, arXiv:2412.13663.

Edwin B. Wilson. 1927. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212.

Ji Xin, Raphael Tang, Yaoliang Lee, and Jimmy Lin. 2021. The art of abstention: Selective prediction and error conditioning in question answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, pages 1040–1051.

Kevin Yang, Dan Klein, Asli Celikyilmaz, Nanyun Peng, and Yuandong Tian. 2023. RLCD: Reinforcement learning from contrast distillation for language model alignment. *Preprint*, arXiv:2307.12950.

Yilun Zhao et al. 2024. FinDVer: Explainable claim verification over long and hybrid-content financial documents. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14739–14752.

Chujie Zheng et al. 2025. ProcessBench: Identifying process errors in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 1009–1024.

Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems 32*.

A 附录

模型与动作接口。表 4 总结本文讨论的三个结构化决策模型。只有 Laya 与 Vela 在本地运行；Jev 定义商业参考点，本文不评测。

原生单动作基线。表 5 报告两个原生动作在 `search_dev` 池上的准确率与平均每次调用延迟。这些数字定义了每个搜索策略所比较的固定策略。两种表示的每次调用成本相近，因此适应度函数使用调用次数，而不是模型特定的延迟项。峰值常驻内存用于记录 CPU-only 执行包络。

互补性总览。图 2 可视化表 1 背后的总修复与净修复计数，包括把 train 选出的方向原样应用到 dev 与 confirm。

修复轮廓。图 3 显示第二个动作的修复率随 Noul 概率分箱的变化，分别给出训练、dev 与 confirm 池。每个点是该箱中 Noul 错误而 Choice 正确的行比例。当箱中至少有八行时印出箱计数。该图支持第 5 节的论断：修复集中在概率中高段，而不是低置信度尾部。

表 4: 本文讨论的三个结构化决策模型。只有 Laya 与 Vela 在本地运行；Jev 定义商业参考点，本文不评测。

模型	接口	训练路线	本文角色
Jev	一次调用返回带类型的 Noul、Choice 与 Score 决策及概率	TypeSafe 称其为 RLCD 与适当评分目标；完整配方未公开	RLCD / System One 家族的参考点；未运行
Laya 0.4.16	ModernBERT-large 编码器加决策头，约 4.21 亿参数；Noul、Choice、Score	上游称配方为 RLCD，使用 log、spherical 与 ranked probability 分数	所有实验中使用的开放 RLCD 模型
Vela 2.0 0.3B	22 层 ModernBERT，768 隐藏维，8,192 token；Noul、Choice、Score、Span、Set	从 Decision-1.0-Kai 的 Choice 路径初始化，任务特定损失与路由数据；未称 RLCD	接口匹配、配方不同的对照模型

表 5: 60 行选择池（`search_dev`，种子 17）上的原生单动作基线。延迟是本地 CPU 上每次调用的均值，仅用于说明两种表示成本大致相同；它不用于适应度函数。

任务	模型	n	Noul 准确率	Choice 准确率	Noul ms	Choice ms	峰值 RSS (MB)
CodeXGLUE	Laya	60	0.450	0.650	2304	2239	2114
CodeXGLUE	Vela	60	0.633	0.667	2437	2341	1931
FinDVer	Laya	59	0.712	0.644	2768	2741	2117
FinDVer	Vela	59	0.458	0.458	2394	2263	1795
ProcessBench	Laya	60	0.100	–	2616	–	2124
ProcessBench	Vela	60	0.100	–	1577	–	1747

校准细节。表 6 报告校准及其与路由的关系；图 4 画出四个二元单元在 confirm 上的可靠性图，包含温度缩放前后。缩放参数在 `search_train` 上拟合，原样应用到 confirm。由于温度缩放是单调的，它改变数值概率，但不改变行的排序，因此也不改变路由器可用的带策略集合。

搜索诊断。图 5 汇总第 5 节讨论的三个轨迹诊断：中性变异率、档案中训练与 dev 适应度的秩相关，以及种子 17 的历史最优训练适应度。

排序与策略价值。图 6 画出每个单元的折外 AUROC 与冻结的 dev 选择策略的 confirm 净效用。位于 0.5 右侧的点有排序信号；位于零以下的点有负策略价值。分离清晰可见：排序信号最强的单元（FinDVer/Vela）confirm 净值为负，而 confirm 净值最好的单元（FinDVer/Laya）排序信号接近随机。

边界检查。表 7 报告迁移计数与四个最高严重度的故障图单元。图 7 显示按任务、模型与置信带划分的完整 confirm 池严重度图。

复现细节。本文所有结果都来自已提交的 JSON 文件。逐行缓存是每个池一个 JSON 文件，命名为 `cache_<split>_<task>_<model>.json`，位于 `results/v4/evolution/` 下。分析脚本包括：

- `scripts/analyze_action_complementarity.py`
- `analyze_complementarity_heldout.py`

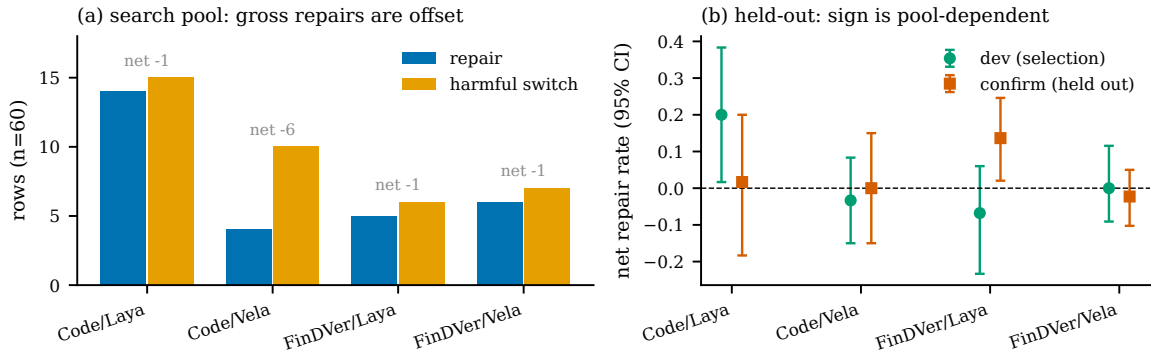


图 2: 动作互补性。(a) 在训练池上，第二个动作修复错误，但也破坏正确答案；每对柱上方印有净计数。(b) 同一 train 选出方向应用到 dev 与 confirm：净修复的符号跨池变化，只有一个 confirm 单元的区间高于零。

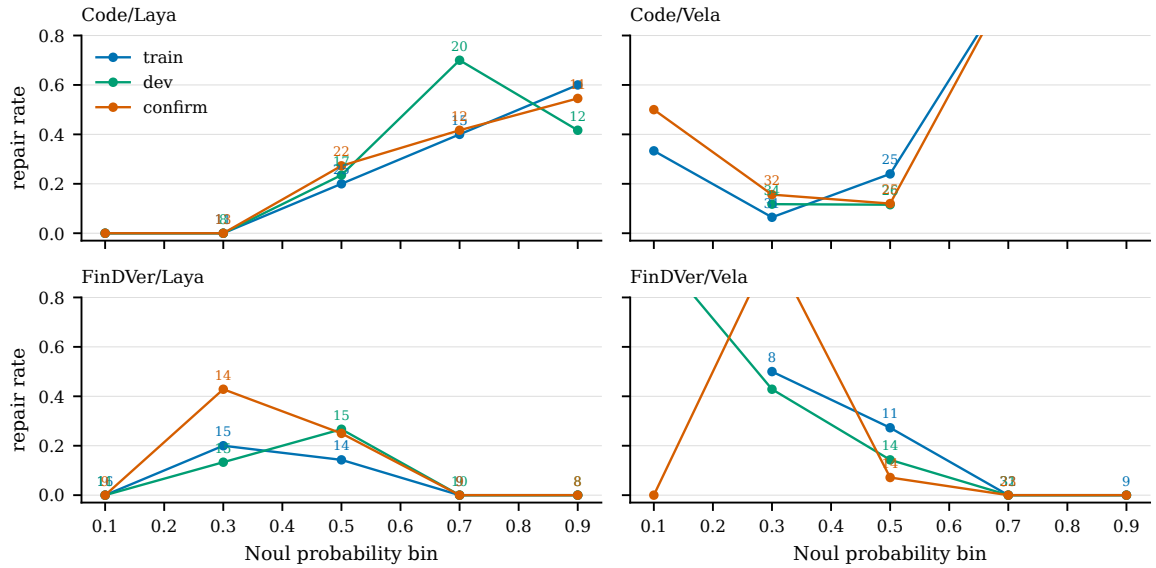


图 3: 四个二元单元的修复率随 Noul 概率分箱的变化。最低箱中修复很少，修复集中在 $p \in [0.4, 0.8]$ 附近，而这正是置信度级联最不可能切换的地方。

- `analyze_utility_predictability.py`
- `analyze_search_landscape.py`
- `analyze_transfer_heldout.py`
- `analyze_calibration.py`
- `analyze_seed_extension.py`
- `analyze_fixed_action_cis.py`
- `analyze_search_dynamics.py`

表由 `scripts/make_paper_tables.py` 生成，图由 `scripts/plot_paper_figures.py` 生成；两个脚本都不含手抄数字。GA/ES/NSGA-II 运行器位于 `src/evolution/`。模型修订、数据集修订、种子与 CPU 线程数记录在每个结果文件的元数据中。五种子扩展使用种子 17、23、41、59 与 73；预算探针使用同一缓存，唯一基因组上限为 48。不使用托管 CI 运行器：每次模型调用都在本地 Windows 机器上执行，缓存输出是本文分析的唯一输入。

表 6: `confirm` 上的校准及其与路由的关系。温度 T 在 `search_train` 上拟合；ECE 使用 15 个等宽箱。金标签 AUROC 是概率排序当前动作正确性的能力；修复 AUROC 是它排序第二个动作可修复行的能力。最后两列是在 `search_dev` 上选择的带策略，分别用原始概率与温度缩放概率得到的净修复。

任务 / 模型	ECE	ECE_T	Brier	Brier_T	$\mathrm{AUROC}_{金}$	$\mathrm{AUROC}_{修}$	路由	路由 $_T$
CodeXGLUE / Laya	0.230	0.076	0.287	0.250	0.550	0.791	-5.0	-1.7
CodeXGLUE / Vela	0.090	0.045	0.260	0.250	0.522	0.444	+0.0	+1.7
FinDVer / Laya	0.290	0.141	0.285	0.246	0.603	0.336	+2.3	+0.0
FinDVer / Vela	0.147	0.169	0.218	0.234	0.605	0.095	-2.3	+0.0

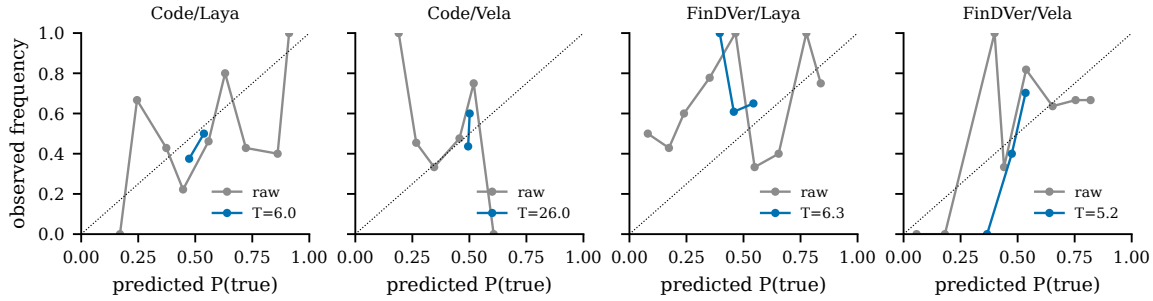


图 4: 四个二元单元在 confirm 池上的可靠性图。灰色为原始 NOUL 概率，蓝色为温度缩放后。温度缩放降低校准误差，但它是单调映射：行的排序、以及在重参数化意义下可用的带策略集合，都不变。

未报告的内容。锁定测试未运行，NOUL 模板未针对替代措辞审计，没有为每个搜索策略测量真实端到端延迟，Jev 未评测。这些省略在正文中说明，没有用代理数字填补。FinDVer/Laya confirm 单元是唯一净修复准确率区间为正的单元；应把它视为更大规模复现的线索，而不是可部署结果。

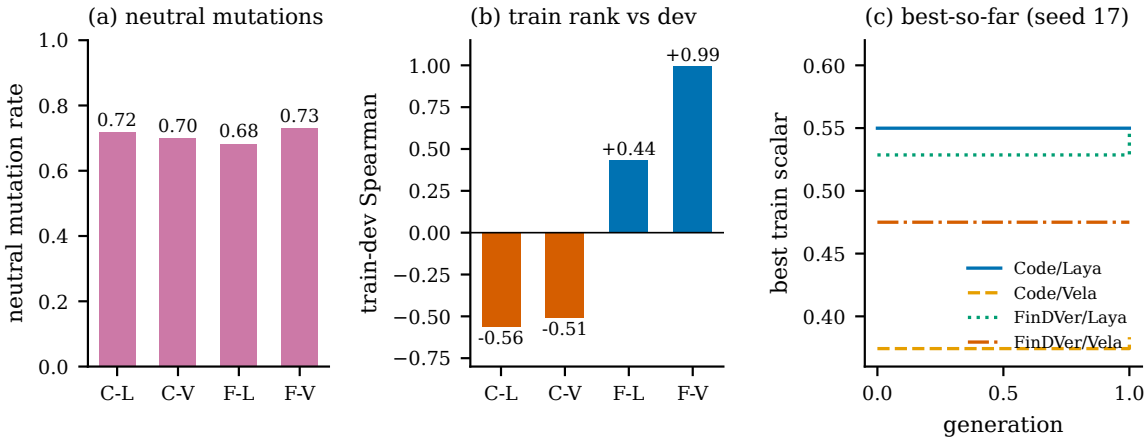


图 5: 搜索诊断。(a) 68% 到 73% 的单基因变异不改变训练适应度。(b) 档案中训练与 dev 适应度的秩相关在两个 CodeXGLUE 单元为负，在 FinDVer 为正。(c) 种子 17 的历史最优训练适应度；大多数运行最多改进一次，若干从第 0 代起就平坦。

表 7: 边界检查。上：冻结基因组迁移，统计相对目标固定仅切换策略的显著配对准确率差异；对角线单元是同任务部署检查，非对角线单元是真正的迁移。下：四个最高严重度的故障图单元，严重度定义为单元内错误率。

迁移	显著正	显著负
对角线（同任务）	0	2
非对角线（真正迁移）	0	4

故障图单元	n discover	严重度	n dev	严重度	n confirm	严重度
ProcessBench / Vela / 中	14	0.929	10	0.800	12	0.833
ProcessBench / Laya / 中	13	0.923	15	0.867	16	0.875
ProcessBench / Vela / 低	45	0.889	46	0.957	47	0.936
ProcessBench / Laya / 高	41	0.756	35	0.914	38	0.895

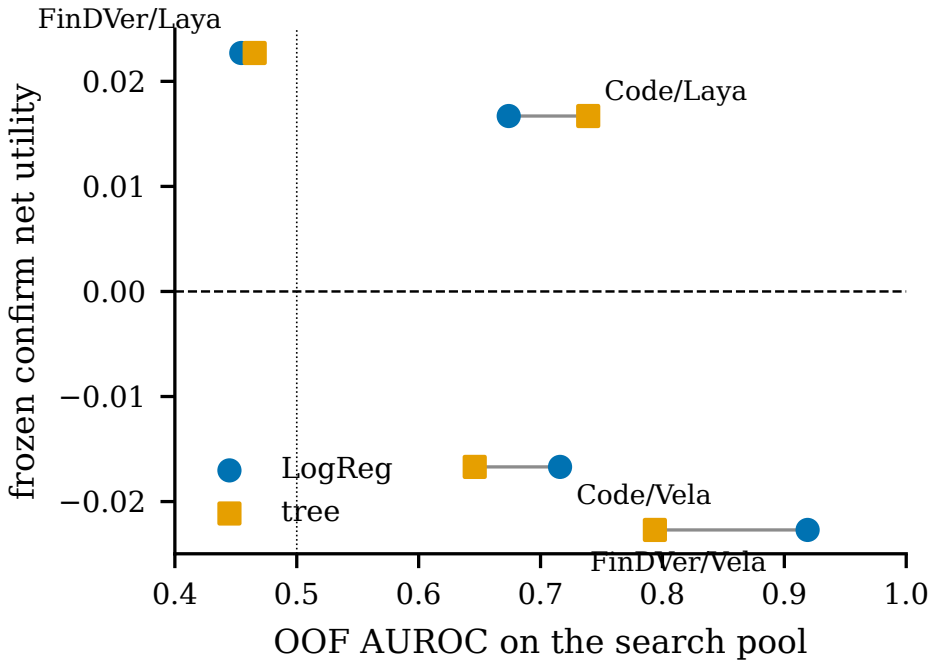


图 6: 折外 AUROC 对 confirm 净效用。水平虚线为零净效用；垂直点线为随机水平 AUROC。排序质量与策略价值不是同一个量。

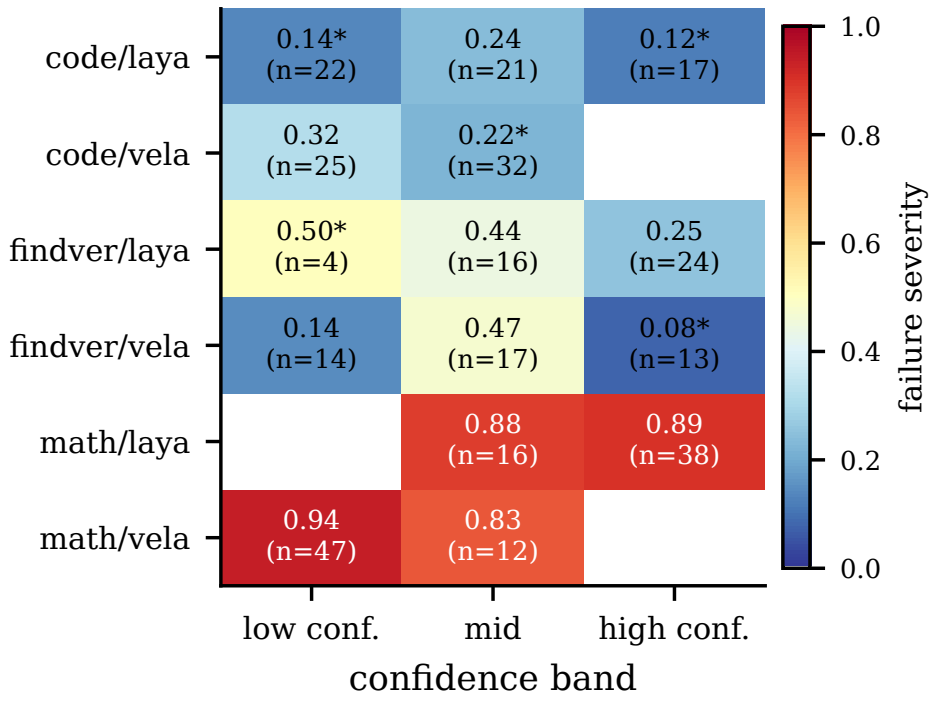


图 7: 按任务、模型与置信带划分的 confirm 池失败严重度。星号标记未在 0.15 容差内确认的单元。稳定的高严重度区域是 ProcessBench 的中置信度带，两种表示在这里一起失败。